Doxygen Demo Project

Generated by Doxygen 1.15.0

1 Doxygen Documentation Demo 1

1.1 Introduction . . . 1

1.2 Features . . . 1

2 Doxygen Documentation 3

2.1 1. Introduction to Doxygen . . . 3

2.1.1 Why is Doxygen used? . . . 3

2.2 2. Doxygen Workflow . . . 3

2.3 3. Doxygen Comment Syntax . . . 4

2.4 4. Common Doxygen Commands . . . 4

2.5 5. Documenting Classes and Functions . . . 4

2.5.1 Example: Documenting a class . . . 4

2.5.2 Documenting function parameters . . . 4

2.6 6. Doxygen Configuration File (Doxyfile) . . . 5

2.7 7. Generating Documentation . . . 5

2.7.1 HTML Documentation . . . 5

2.7.2 PDF Documentation . . . 5

2.8 8. Assignment 9 – Doxygen . . . 5

2.8.1 Objective . . . 5

2.8.2 Files provided . . . 6

2.8.3 Tasks performed . . . 6

2.8.4 Submission requirements . . . 6

2.9 9. What I Learned . . . 7

2.10 10. References . . . 7

3 Todo List 9

4 Class Index 11

4.1 Class List . . . 11

5 File Index 13

5.1 File List . . . 13

6 Class Documentation 15

6.1 CalculationEngine Class Reference . . . 15

6.1.1 Detailed Description . . . 15

6.1.2 Constructor & Destructor Documentation . . . 15

6.1.2.1 CalculationEngine() . . . 15

6.1.3 Member Function Documentation . . . 16

6.1.3.1 power() . . . 16

6.2 Logger Class Reference . . . 16

6.2.1 Detailed Description . . . 16

6.2.2 Constructor & Destructor Documentation . . . 17

6.2.2.1 Logger() . . . 17

6.2.3 Member Function Documentation . . . 17
6.2.3.1 log() . . . 17

7 File Documentation **19**

7.1 CalculationEngine.h File Reference . . . 19
7.1.1 Detailed Description . . . 19
7.2 CalculationEngine.h . . . 19
7.3 Logger.h File Reference . . . 20
7.3.1 Detailed Description . . . 20
7.3.2 Enumeration Type Documentation . . . 20
7.3.2.1 LogLevel . . . 20
7.4 Logger.h . . . 20
7.5 main.cpp File Reference . . . 21
7.5.1 Detailed Description . . . 21
7.5.2 Function Documentation . . . 21
7.5.2.1 main() . . . 21
7.6 README.md File Reference . . . 21

Chapter 1

Doxygen Documentation Demo

1.1 Introduction

This project demonstrates how Doxygen generates documentation for a multi-file C++ project.

1.2 Features

- Logging using the Logger class.
- Mathematical calculations using CalculationEngine.
- Calculation of powers of numbers.

Chapter 2

Doxygen Documentation

2.1 1. Introduction to Doxygen

Doxygen is an open-source documentation generator used to create documentation directly from source code.

It extracts information from specially formatted comments written in the code and generates reference documentation.

It supports programming languages such as C, C++, Python and Java.

2.1.1 Why is Doxygen used?

- Automatically generates documentation from source code.
- Makes code easier to understand and maintain.
- Documents classes, functions, parameters and return values.
- Generates HTML and PDF documentation.

2.2 2. Doxygen Workflow

The basic workflow for using Doxygen is:

1. Write specially formatted comments in the source code.
2. Create or edit the Doxyfile configuration.
3. Run Doxygen using the configuration file.
4. Open the generated documentation.

```
doxygen -g Doxyfile
doxygen Doxyfile
```

The first command creates a configuration file named `Doxyfile`. The second command generates the documentation using that configuration.

2.3 3. Doxygen Comment Syntax

Doxygen uses specially formatted comments to identify documentation.

A common format in C++ is:

```
int myFunction(int x);
```

The `/** ... */` format tells Doxygen that the comment contains documentation.

2.4 4. Common Doxygen Commands

Command	Purpose
`@brief`	Provides a short description
`@param`	Describes a function parameter
`@return`	Describes the return value
`@author`	Specifies the author
`@version`	Specifies the version
`@see`	Refers to related documentation
`@note`	Adds an additional note
`@class`	Documents a class
`@enum`	Documents an enumeration
`@file`	Documents a source file
`@mainpage`	Creates the main project documentation page
`@detail`	Adds a detailed description

Both `@` and `\` can be used to write Doxygen commands. For example, `@brief` and `\brief` have the same purpose.

2.5 5. Documenting Classes and Functions

Doxygen can document classes, their constructors, member functions and other elements.

2.5.1 Example: Documenting a class

```
class Car {
public:
    bool startEngine();
};
```

2.5.2 Documenting function parameters

The `@param` command describes the inputs passed to a function.

The `@return` command describes the value returned by the function.

```
int power(int base, int exponent);
```

2.6 6. Doxygen Configuration File (Doxyfile)

The Doxyfile is the configuration file that controls how Doxygen generates documentation.

It contains settings such as the project name and the output formats.

For the assignment, the configuration had to generate both HTML and LaTeX output.

Important settings:

```
GENERATE_HTML = YES
GENERATE_LATEX = YES
GENERATE_RTF = NO
GENERATE_XML = NO
```

The provided Doxyfile template can be used as the starting point and modified as required.

2.7 7. Generating Documentation

After completing the Doxygen comments and configuring the Doxyfile, run:

```
doxygen Doxyfile
```

This generates the documentation.

2.7.1 HTML Documentation

The generated HTML documentation can be viewed by opening:

```
html/index.html
```

It provides a browser-based view of the documented project.

2.7.2 PDF Documentation

To generate a PDF:

1. Navigate to the generated `latex` directory.
2. Run the command:

```
make
```

This generates the PDF documentation from the LaTeX output.

2.8 8. Assignment 9 – Doxygen

2.8.1 Objective

Create documentation for a small multi-file C++ project using Doxygen.

2.8.2 Files provided

The assignment provided the following files:

- `Logger.h` – Defines a simple Logging class.
- `CalculationEngine.h` – Defines a class for mathematical operations that uses the Logger.
- `main.cpp` – Contains the application entry point.
- `Doxyfile` template – Provides a starting configuration for generating documentation.

2.8.3 Tasks performed

1. Install Doxygen

On Ubuntu, Doxygen can be installed using:

```
sudo apt install doxygen
```

2. Complete the source code documentation

Find the TODO comments and undocumented elements in the provided source files. Add correctly formatted Doxygen comments.

The assignment specifically required:

- **Main page:** Use `\mainpage` or `@file` to define the project overview.
- **Logger class:** Use `@file`, `@class` and `@brief`.
- **Log Level enum:** Use `@enum` and document every enumerator value.
- **Logger constructor:** Document the constructor and its parameters.
- **CalculationEngine class:** Use `@file`, `@class`, `@brief` and `@detail`.
- **CalculationEngine::power:** Add `@param` descriptions for `base` and `exponent`, and an `@return` description. Review the existing `\todo` tag.

3. Generate HTML and PDF documentation

- Configure the Doxyfile to generate HTML and LaTeX output.
- Run `doxygen Doxyfile` to generate the documentation.
- Open `html/index.html` to view the HTML documentation.
- Navigate to the generated LaTeX directory and run `make` to generate the PDF.

2.8.4 Submission requirements

The following files had to be uploaded to Google Classroom:

1. Three completed source files.
2. Final Doxyfile used for generation.
3. Screenshot of the generated HTML documentation.
4. Generated PDF documentation.

2.9 9. What I Learned

Through this assignment, I learned how to:

- Write specially formatted Doxygen comments in C++.
- Document classes, constructors, enumerations and functions.
- Describe function parameters and return values.
- Configure Doxygen using a Doxyfile.
- Generate HTML and PDF documentation from source code.
- Use automated documentation to make a project easier to understand and maintain.

2.10 10. References

- Introduction to Documentation Tools: Doxygen & Google Docs – Lecture slides.
- Assignment 9 – Doxygen.
- Official Doxygen Manual: https://www.doxygen.nl/manual/

Chapter 3

Todo List

Member CalculationEngine::power (double base, int exponent)

Add support for additional mathematical operations.

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CalculationEngine
Performs mathematical operations 15
Logger
Provides basic logging functionality 16

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

CalculationEngine.h
: Defines the CalculationEngine class 19

Logger.h
: Defines a simple logging system 20

main.cpp
: Main entry point of the documentation demo project 21

Chapter 6

Class Documentation

6.1 CalculationEngine Class Reference

Performs mathematical operations.

`#include <CalculationEngine.h>`

Public Member Functions

- CalculationEngine (Logger &logger)

  Constructs a CalculationEngine object.
- double power (double base, int exponent)

  Calculates the power of a number.

6.1.1 Detailed Description

Performs mathematical operations.

This class performs calculations and uses the Logger class to record information about its operations.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 CalculationEngine()

```
CalculationEngine::CalculationEngine (
            Logger & logger)
```

Constructs a CalculationEngine object.

Parameters

logger	Reference to the Logger object.

6.1.3 Member Function Documentation

6.1.3.1 power()

```
double CalculationEngine::power (
            double base,
            int exponent)
```

Calculates the power of a number.

Parameters

base	The base number.
exponent	The exponent.

Returns

The result of raising base to exponent.

Todo Add support for additional mathematical operations.

The documentation for this class was generated from the following file:

- CalculationEngine.h

6.2 Logger Class Reference

Provides basic logging functionality.

`#include <Logger.h>`

Public Member Functions

- Logger (LogLevel level)
  Constructs a Logger object.
- void log (const char *message)
  Logs a message.

6.2.1 Detailed Description

Provides basic logging functionality.

The Logger class displays messages with their corresponding severity levels.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 Logger()

```
Logger::Logger (
            LogLevel level)
```

Constructs a Logger object.

Parameters

level	The minimum logging level.

6.2.3 Member Function Documentation

6.2.3.1 log()

```
void Logger::log (
            const char * message)
```

Logs a message.

Parameters

message	The message to be displayed.

The documentation for this class was generated from the following file:

- Logger.h

Chapter 7

File Documentation

7.1 CalculationEngine.h File Reference

Defines the CalculationEngine class.

```
#include "Logger.h"
```

Classes

- class CalculationEngine

  Performs mathematical operations.

7.1.1 Detailed Description

Defines the CalculationEngine class.

7.2 CalculationEngine.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef CALCULATION_ENGINE_H
00007 #define CALCULATION_ENGINE_H
00008
00009 #include "Logger.h"
00010
00018 class CalculationEngine {
00019 public:
00020
00025     CalculationEngine(Logger& logger);
00026
00036     double power(double base, int exponent);
00037
00038 private:
00039     Logger& logger;
00040 };
00041
00042 #endif
```

7.3 Logger.h File Reference

Defines a simple logging system.

Classes

- class Logger

  Provides basic logging functionality.

Enumerations

- enum LogLevel { DEBUG , INFO , WARNING , ERROR }

  Defines the severity levels for log messages.

7.3.1 Detailed Description

Defines a simple logging system.

This file contains the Logger class and LogLevel enum.

7.3.2 Enumeration Type Documentation

7.3.2.1 LogLevel

```
enum LogLevel
```

Defines the severity levels for log messages.

Enumerator

DEBUG	Detailed debugging information.
INFO	General informational messages.
WARNING	Warning messages about potential issues.
ERROR	Error messages indicating failures.

7.4 Logger.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef LOGGER_H
00009 #define LOGGER_H
00010
00015 enum LogLevel {
00016     DEBUG,
00017     INFO,
00018     WARNING,
00019     ERROR
00020 };
00021
00029 class Logger {
00030 public:
00031
00036     Logger(LogLevel level);
00037
00042     void log(const char* message);
00043
00044 private:
00045     LogLevel level;
00046 };
00047
00048 #endif
```

7.5 main.cpp File Reference

Main entry point of the documentation demo project.

```
#include <iostream>
#include "Logger.h"
#include "CalculationEngine.h"
```

Functions

- int main ()

7.5.1 Detailed Description

Main entry point of the documentation demo project.

This program demonstrates the use of Logger and CalculationEngine classes.

7.5.2 Function Documentation

7.5.2.1 main()

```
int main ()
```

7.6 README.md File Reference